COMP3320/6464 High-Performance Scientific Computing

OpenMP SIMD

Dr Haibo Zhang
School of Computing
Australian National University

Logistics

- **Assignment 2 is due in 10 days !!**
  - Ask questions in Ed if you are facing challenges
- Final exam scheduled on November 12
  - Start preparation as early as possible
  - Ask questions in Ed
- Schedule for the remaining lectures
  - This week
    - Vectorization with OpenMP
    - Analysis of parallel scalability
  - Next week
    - Special topic on photonic computing
    - Wrap-up & final exam

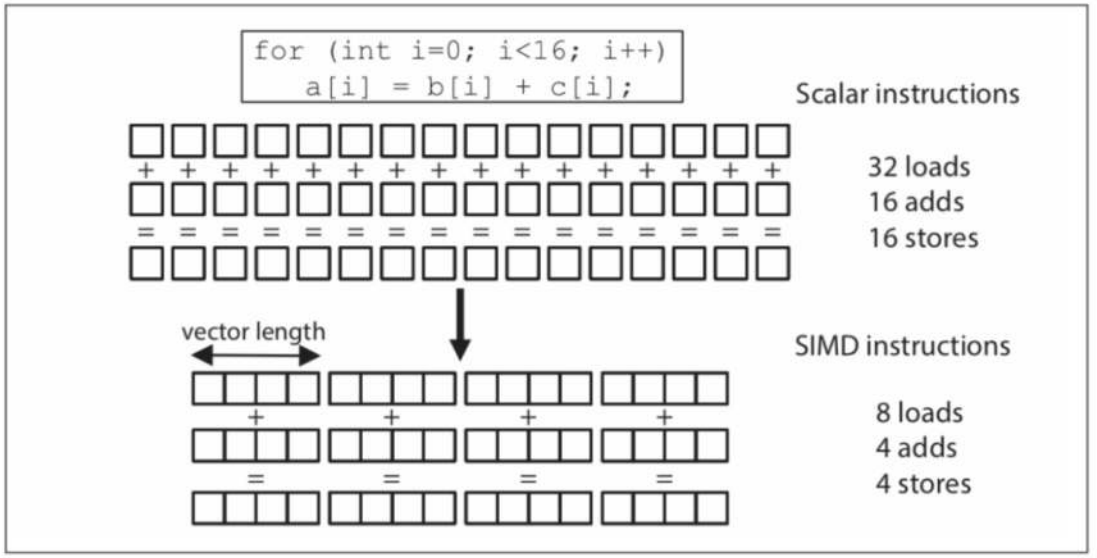
OpenMP Reference Material

- **Using OpenMP – The Next Step**, R. van der Pas, E. Stotzer, and C. Terboven, Chapter 5
- http://www.openmp.org/

OpenMP SIMD

Australian National University

- SIMD provides data-parallelism at the instruction level
- A single instruction operates on multiple data elements in parallel
- SIMD instructions use special registers a larger width (*vector length*)
- SIMD instructions are as fast as their scalar counterpart, leading to potential speedup of up to the vector length
- In practice, the speedup achieved may depend heavily on memory operations needed to move the data



Challenges for compiler vectorization

The compiler can identify operations suitable for vectorization (or you can give it a hint using OpenMP directives), but it must prove it legal to do so. The following issues tend to prevent the compiler from vectorizing your code:

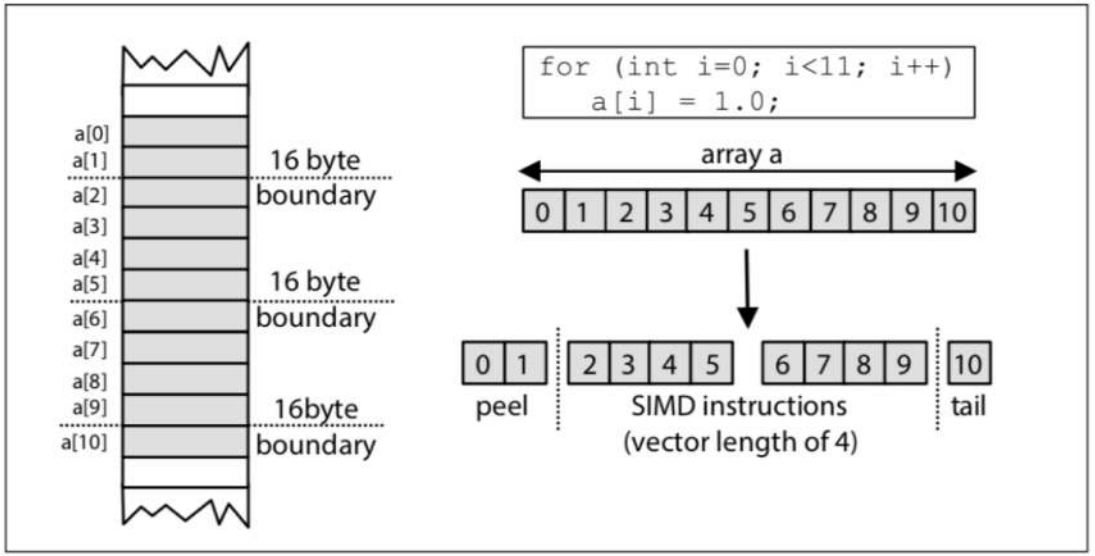
- Imprecise dependence information (potential pointer aliasing)
- Bad data layout and alignment
- Branching
- Calls to functions
- Loop bounds that are not multiples of SIMD length

Alignment

Australian National University

To get full benefit from SIMD the starting address of the vectors (arrays) needs to be aligned on a correct memory address boundary

- Starting array address in memory must be a multiple of of the SIMD length
- No optimal alignment → compiler has use *loop peeling*
- Loop peeling generates a loop peel and (usually) a loop tail that are treated separately
- The peel and tail section are typically not executed using SIMD instruction



The simd construct

Australian National University

```
#pragma omp simd [clause_list]
  for-loop
```

- The OMP compiler may transform a loop marked with the `simd` construct into a SIMD loop.
- A *SIMD chunk* of iterations (equal to the SIMD length) is executed by a single thread
- Within a chunk each iteration is executed by a *SIMD lane*
- The following clauses are supported on the `simd` construct

private *(list)*
lastprivate *(list)*
reduction *(reduction-identifier : list)*
collapse *(n)*
simdlen *(length)*
safelen *(length)*
linear *(list[:linear-step])*
aligned *(list[:alignment])*

The simd construct


```
#pragma omp simd [clause_list]
  for-loop
```

- If you use the `simd` construct, you are instructing the compiler to use SIMD instructions
- In case there is pointer aliasing your code will give incorrect results
- In case you do not provide the SIMD length through the `simdlen` clause, the compiler will select an appropriate vector length
- Similarly to the `for` construct the compiler will create a new instance of the loop variable *i* for each SIDM lane.

```
1 void simd_loop(double *a, double *b, double *c, int n)
2 {
3   int i;
4
5   #pragma omp simd
6   for (i=0; i<n; i++)
7      a[i] = b[i] + c[i];
8 }
```

The simdlen and safelen clauses

Australian National University

```
#pragma omp simd simdlen(SIMD_length) safelen(safe_length)
  for-loop
```

- `simdlen` specifies the preferred number of iterations to be executed concurrently
- The value in the clause is a preference. The compiler has the freedom to deviate from this choice and to choose a different length.
- A limit on the vector length used by SIMD instructions is sometimes required. An example of this is when vectorizing a loop with loop-carried dependences.
- `safelen` specifies the loop-carried dependence distance. It is an an upper limit on the vector length.
- If the clause is not specified, the assumed value for `safelen` is the number of loop iterations.

The simd construct

Australian National University

```
#pragma omp simd [clause_list]
  for-loop
```

- The `reduction` and `collapse` clauses works for SIMD loop as well!
- For each variable in the `reduction` list, a private instance is used during the execution of the SIMD loop, with all private instances being combined using the reduction operator
- When using `collapse` use the compiler commentary to check that vectorization actually was performed

```
1 void simd_loop_collapse(double *r, double *b, double *c,
2                         int n, int m)
3 {
4   int i, j;
5   double t1;
6
7   t1 = 0.0;
8   #pragma omp simd reduction(+:t1) collapse(2)
9   for (i = 0; i<n; i++)
10      for (j = 0; j<m; j++)
11          t1 += func1(b[i], c[j]);
12  *r = t1;
13 }
```

The aligned clause

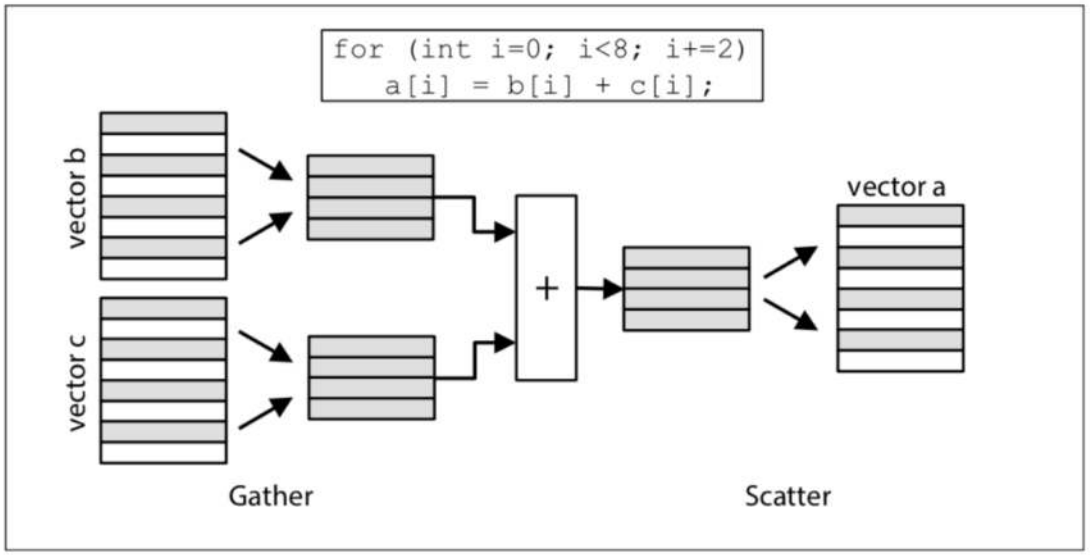

```
#pragma omp simd aligned(list[:alignment])
  for-loop
```

- For compiler optimizations if you arrays have been allocated at optimal alignment boundaries (e.g. `aligned_alloc`, `memalign`, `posix_memalign`)
- In C, a variable that appears in the clause must have an array or pointer type. In C++, a variable that appears in the clause must have array, pointer, reference to array, or reference to pointer type.

Gather/scatter data in/out of vectors

Australian National University

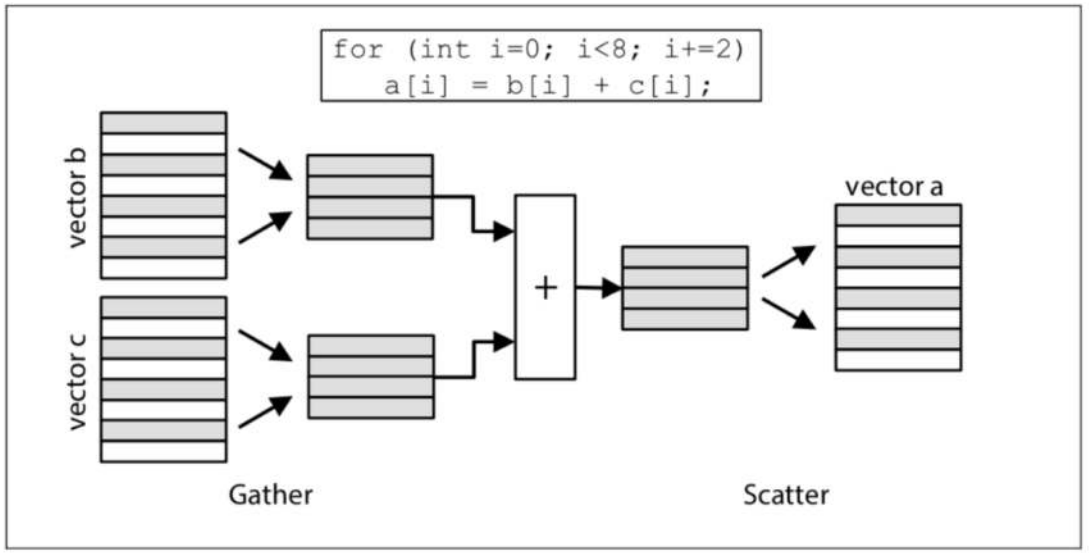
- Scalar data elements are packed into vectors, operated on collectively as a vector by SIMD instructions, and then unpacked.
- When accessing the scalar data with stride 1 (assuming optimal alignment) a single SIMD load/store instruction can be used for packing/unpacking.
- When accessing the scalar data with stride > 1, the compiler will need to write code to perform gather and scatter operations for vector packing/unpacking



Gather/scatter data in/out of vectors

Australian National University

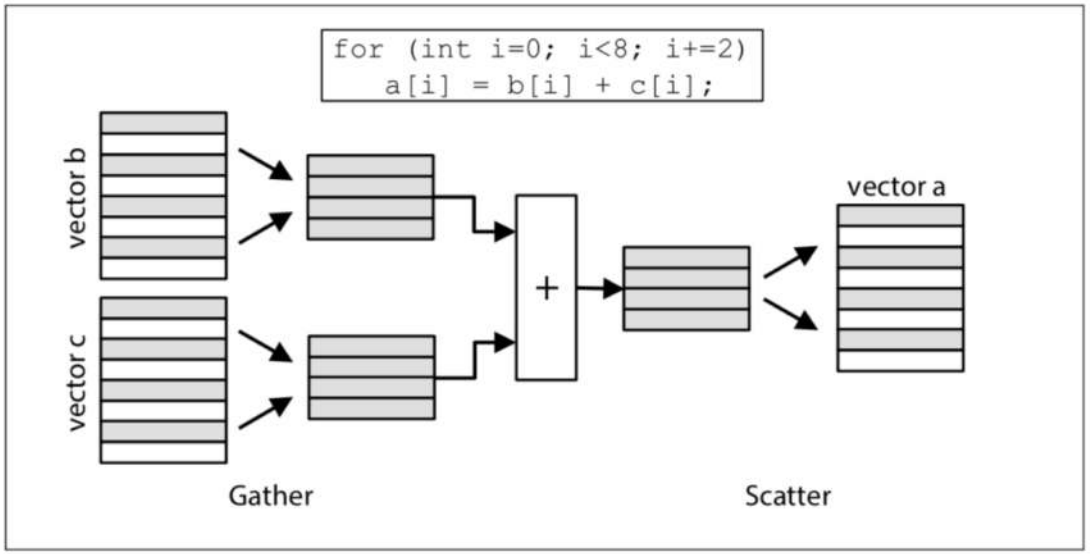
- A gather operation reads scalar data elements from memory linearly but with a stride greater than one.
- A scatter operation writes the scalar data elements in a vector back to memory linearly with a stride greater than one.
- Some architectures support SIMD gather and scatter instruction.



Gather/scatter data in/out of vectors

Australian National University

- In general gather/scatter operations are much more expensive than single vector load/store instruction (best scenario)
- The next best scenario is when the access pattern is linear but with a stride that is greater than one and gather and scatter instructions may be used.
- The worst case scenario is when no linear access pattern can be determined and the scalar data elements must be individually packed into and unpacked from vectors.



The `linear` clause

```
#pragma omp simd linear(list[:linear-step])
  for-loop
```

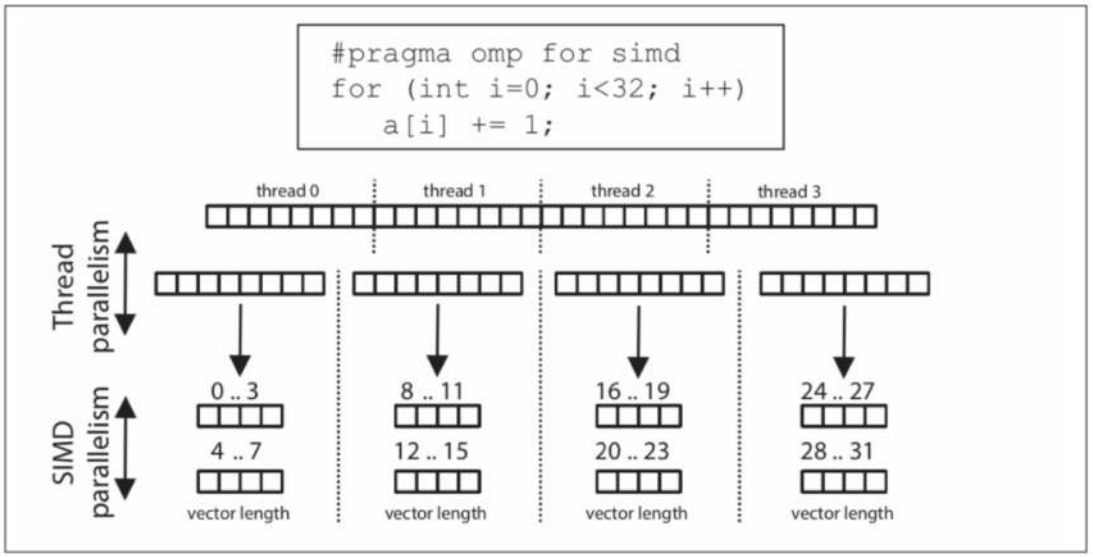
- The `linear` clause is used to indicate the linear behavior of a variable in a loop.
- Compiler uses this information to determine the most efficient way to pack and unpack scalar data elements into vectors.
- The `linear` clause accepts a list of variables as its argument. An optional, colon-separated, linear-step (stride) is supported.

```
1 void simd_loop_linear(double *a, double *b, double *c, int n,
2                        int offset)
3 {
4   int i, j = 0;
5
6   #pragma omp simd linear(j:1)
7   for (i=offset; i<n; i+=2)
8     a[i] = b[j++] + c[i];
9 }
```

The composite `for simd` construct


```
#pragma omp for simd [clause_list]
  for-loop
```

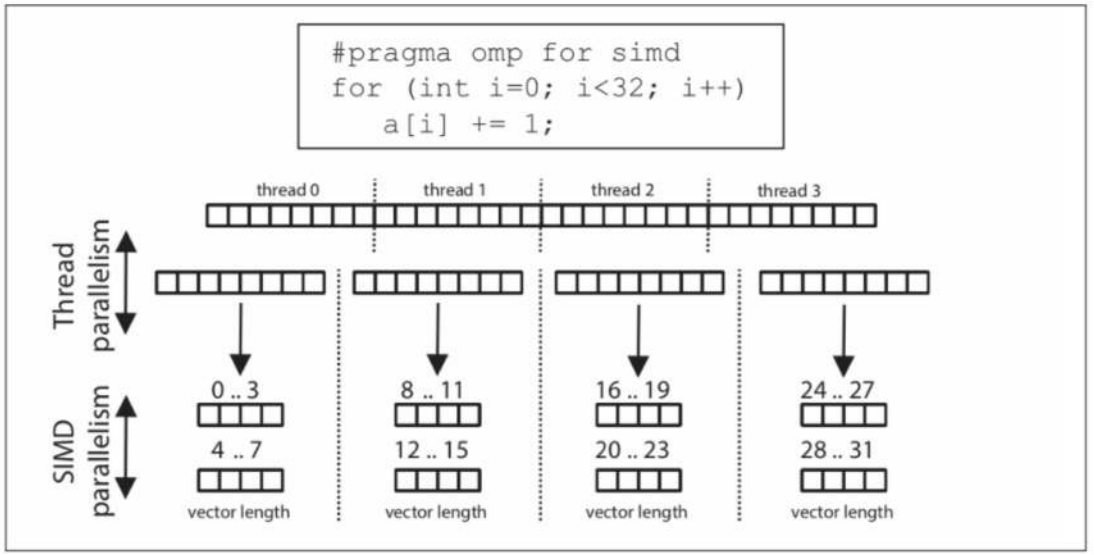
- Chunks of loop iterations are first distributed across the threads in a team according to the clauses on the `for` directive (e.g. `schedule`)
- Each chunk of loop iterations may be converted into SIMD loops in a way that is determined by any clauses that apply to the `simd` construct



The composite `for simd` construct

```
#pragma omp for simd [clause_list]
  for-loop
```

- There may be a significantly increased memory pressure due to the fact that at runtime a new copy of each private variable will be allocated per SIMD lane.
- Typically, the chunks performed with SIMD needs to be large enough to make the loop peel/tail overhead negligible.



The composite `for simd` construct


```
#pragma omp for simd [clause_list]
  for-loop
```

- Remember that you can use the `simd` schedule modifier.
- The modifier will adjust the chunk size according to the formula $(\lceil chunk\_size/SIMD\_len \rceil \times SIMD\_len)$
- This ensures that the size of chunk is at least as large as the SIMD length.

```
10 void func_2(float *a, float *b, int n)
11 {
12   #pragma omp for simd schedule(simd:static, 5)
13   for (int k=0; k<n; k++)
14   {
15     // do some work on a and b
16   }
17 }
```